

Introduction

Méthode

Code

Conclusion

FILESYSTEM WORK-STEALING CRAWLER

Triozon Lucas

–

Guyon Noah

LES ALGORITHMES DE CRAWLING



OBJECTIF :

- Retrouver des fichiers de manière efficace
- Ne pas parcourir toute l'arborescence
- Permettre au workers de ne pas rester inactifs

ARBORESCENCE DE FICHIERS ET DOSSIERS

- Pour tester les algorithmes.
- Arborescence récursive de dossiers imbriqués.
- Fichiers créés avec du texte de longueur aléatoire.



Cahier des charges

- Avoir un crawler fonctionnel
- Avec des workers utilisant leurs piles
- Les piles doivent être “volables”
- Tout doit être parallélisable

Notre travail a été réalisé de manière itérative pour implémenter les différents points :

Crawler de base

Système de
workers basique
avec des threads

Work-stealing

Parallélisation avec
des process

Crawler simple

- Reproduit le comportement de `os.walk()`
- Ouvre un dossier un dossier racine, récupère tout le contenu et l'ajoute à la pile
- A chaque dépileage :
 - Vérifie si ce qu'on cherche se trouve dans l'élément
 - Si on croise un dossier, l'analyse et ajoute le contenu à la pile
- Une seule pile

```
rel_path = stack.pop()
full_path = os.path.join(path_root, rel_path)

if os.path.basename(p=full_path).lower() == search:
    found.append(full_path)
elif not os.path.isfile(path=full_path):
    try:
        children = os.listdir(full_path)
        stack.extend(map(lambda x:os.path.join(rel_path, x), children))
    except PermissionError:
        continue
```

Workers basés sur des Threads

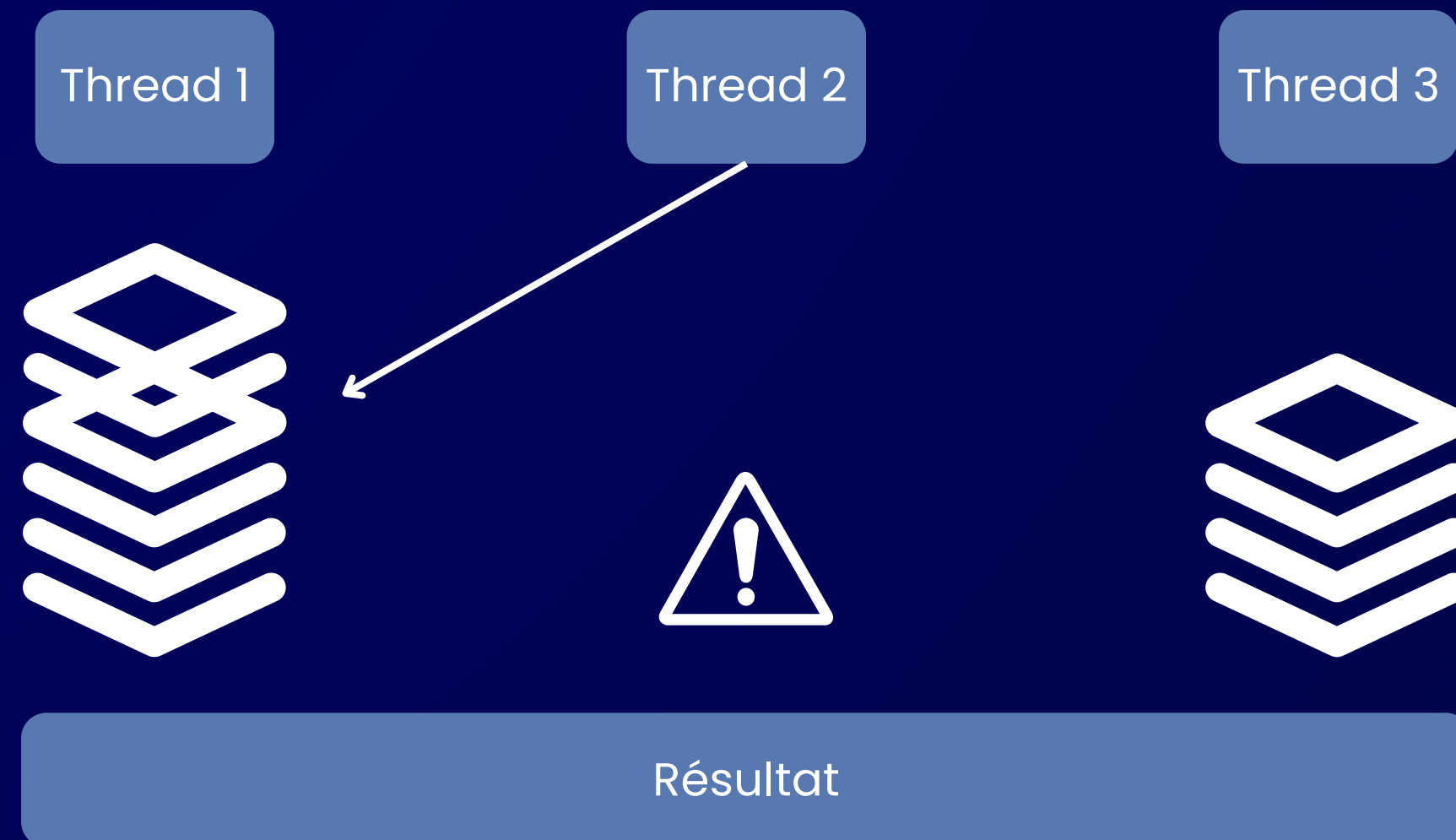
- Nécessite l'initialisation des piles avant le démarrage des threads
- Toutes les piles sont visibles à tous, chaque worker est assigné à une pile spécifique
- Chaque worker exécute le crawler sur sa pile



Work-stealing

```
if len(stacks[i]) == 0:  
    lens = list(map(lambda x:len(x), stacks))  
    j = lens.index(max(lens))  
    stacks[j], stacks[i] = stacks[j][:len(stacks[j])//2], stacks[j][len(stacks[j])//2:]
```

- Quand un worker n'a plus rien à faire, il part observer les autres piles
- Récupère la moitié du contenu de la pile la plus haute
- Et reprend son exécution



Cahier des charges

- Avoir un crawler fonctionnel ✓
- Avec des workers utilisant leurs piles ✓
- Les piles doivent être “volables” ✓
- Tout doit être parallélisable ✗

Crawler de base

Système de
workers basique
avec des threads

Work-stealing

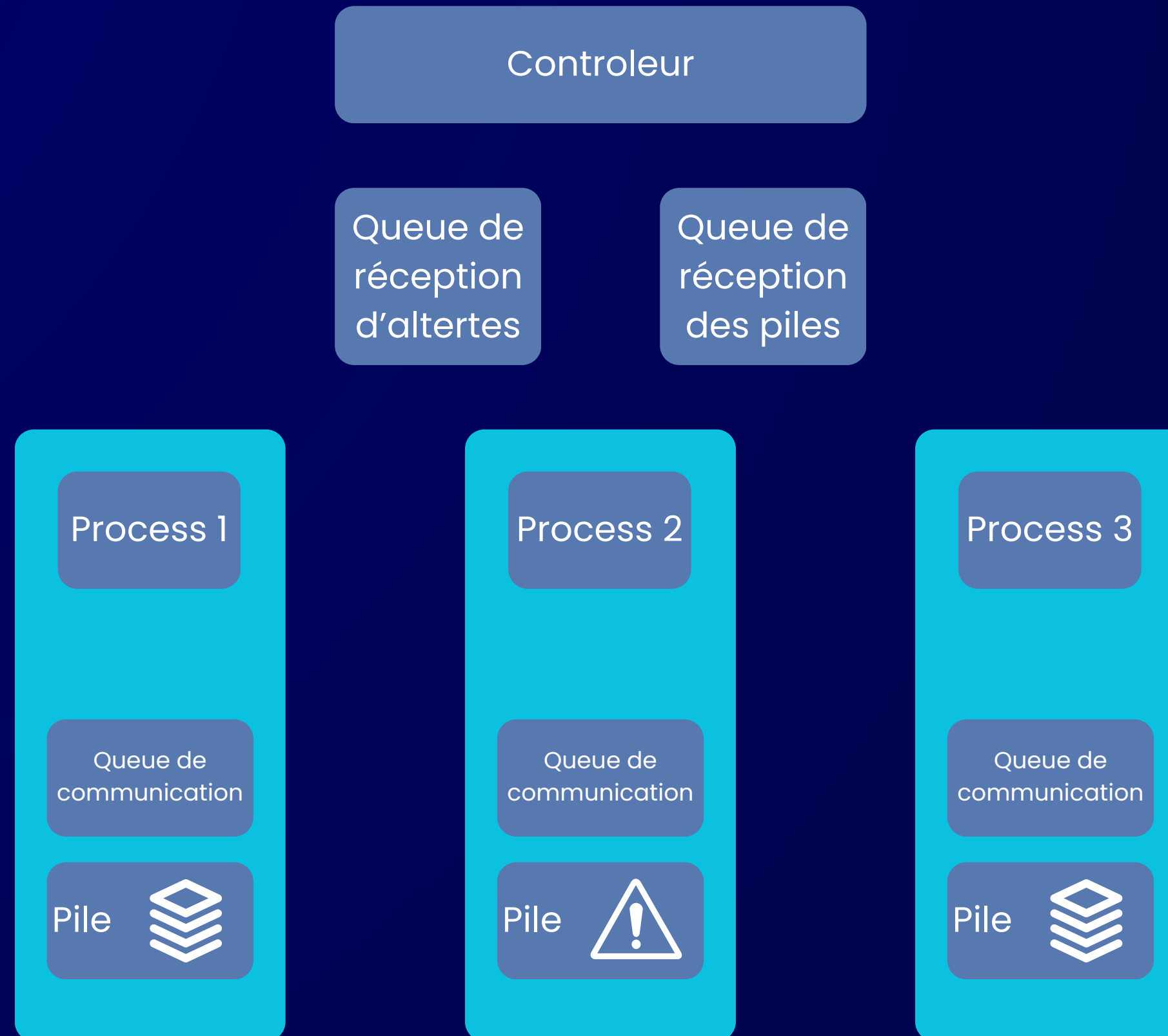
Parallélisation avec
des process

Multiprocessing

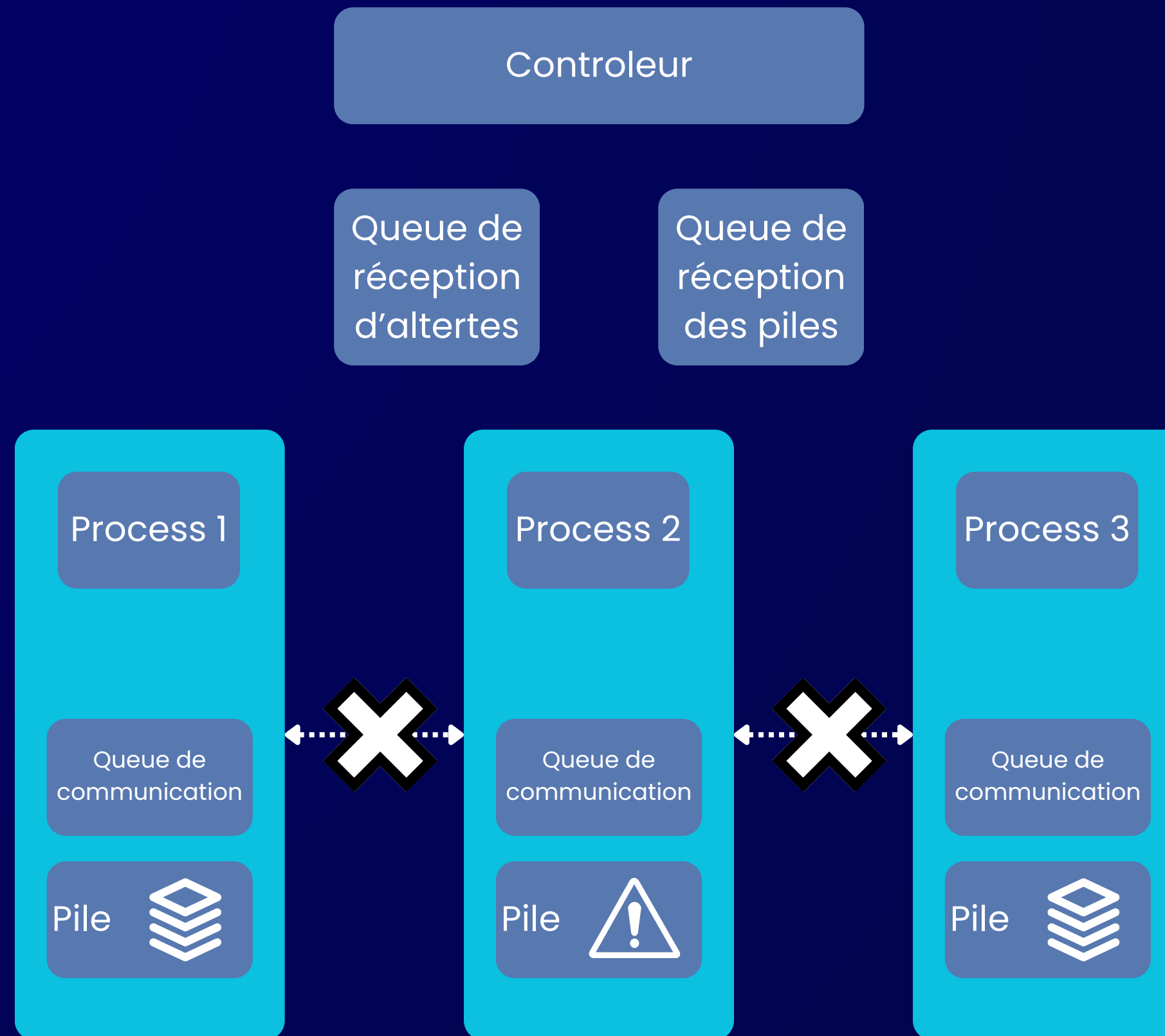
- Les piles ne peuvent plus être partagées
- Alternative : utiliser `Manager.List()`, mais opérations 1000x plus lentes que des listes
- Idée :
 - Les piles seront locales aux processus
 - Les processus vont communiquer formellement entre eux pour demander la pile des autres

Multiprocessing

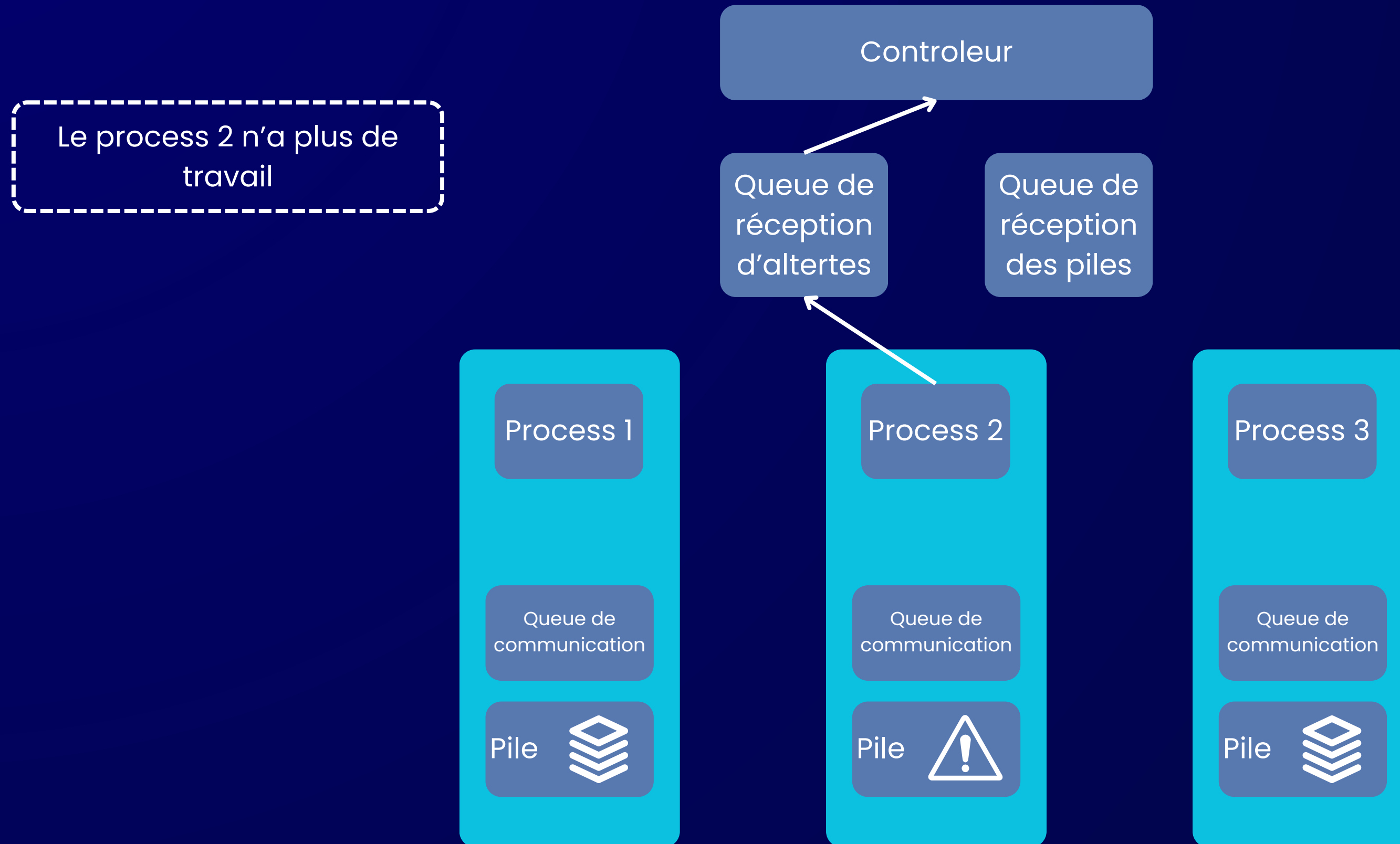
Page 09



Multiprocessing

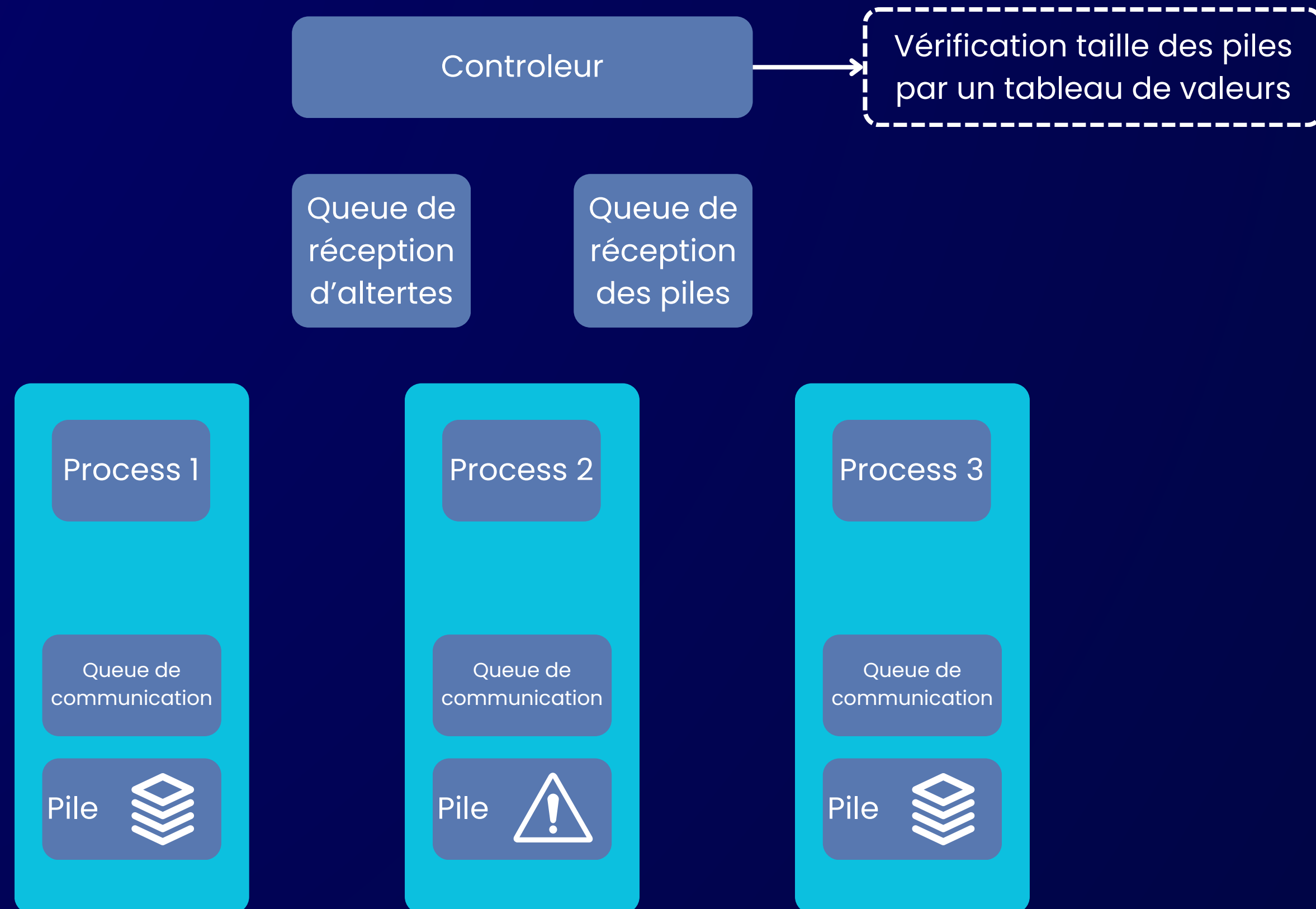


Multiprocessing



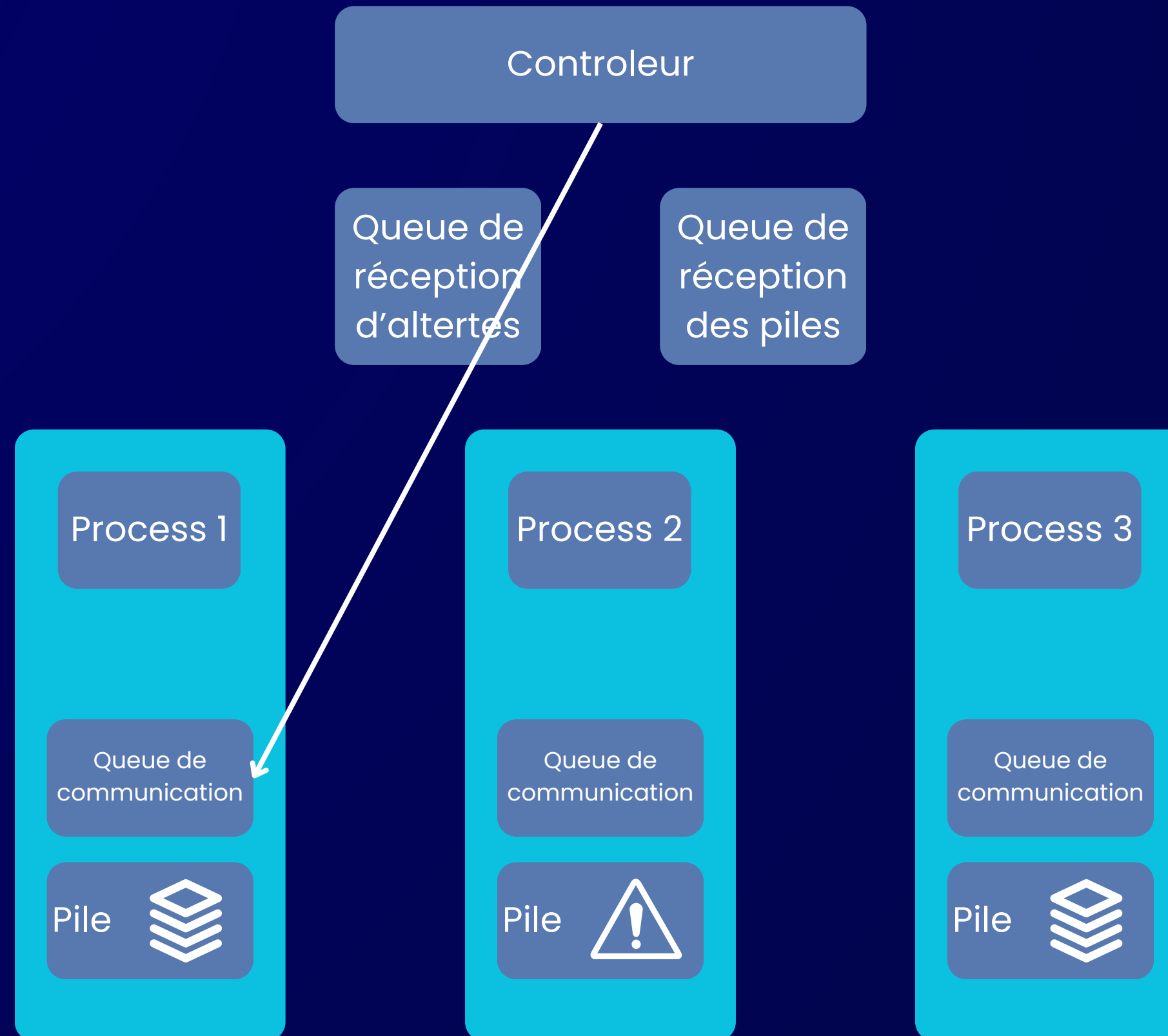
Multiprocessing

Page 09



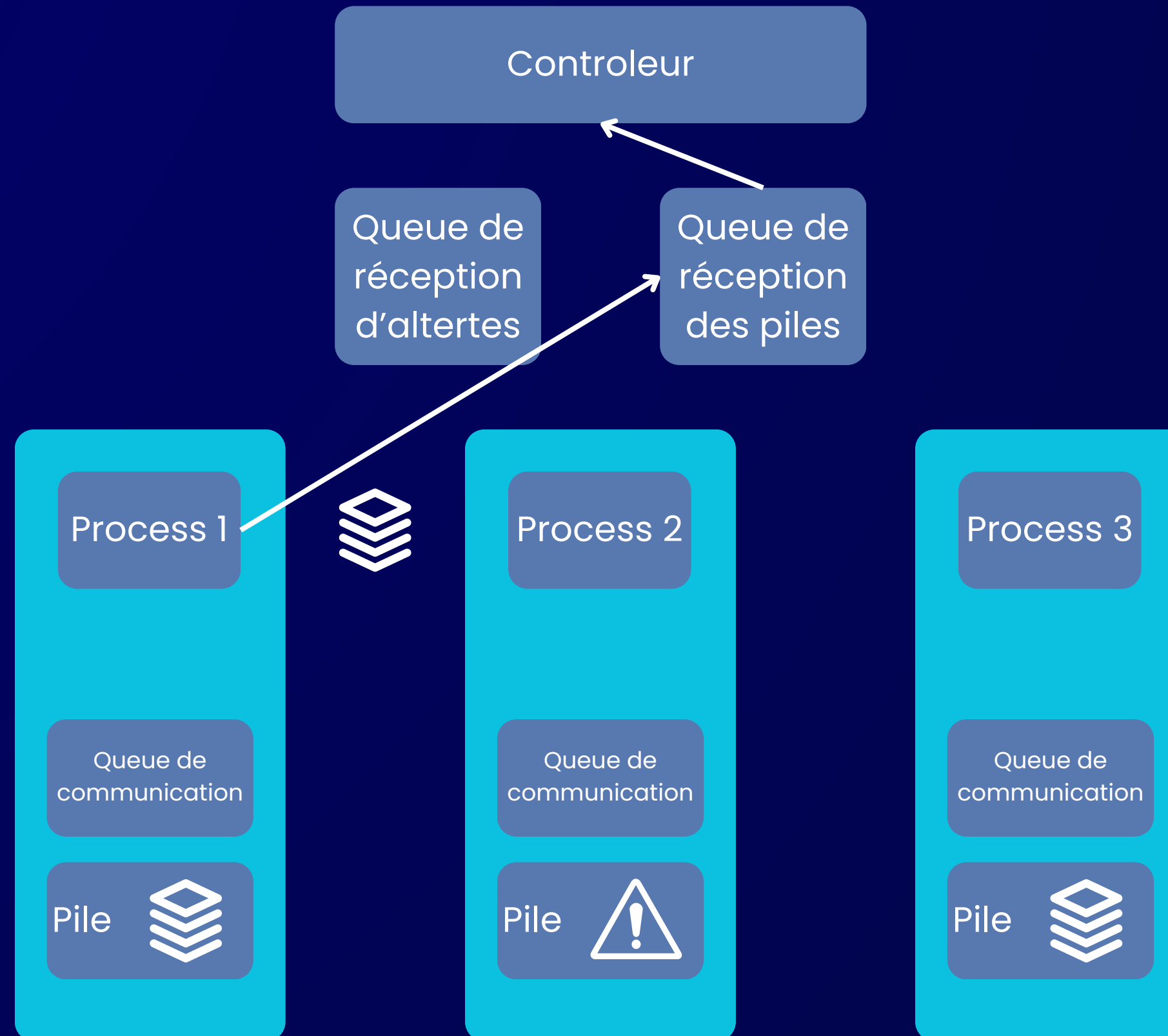
Multiprocessing

Le process 1 est celui qui est le plus occupé, le contrôleur lui demande la moitié de sa pile



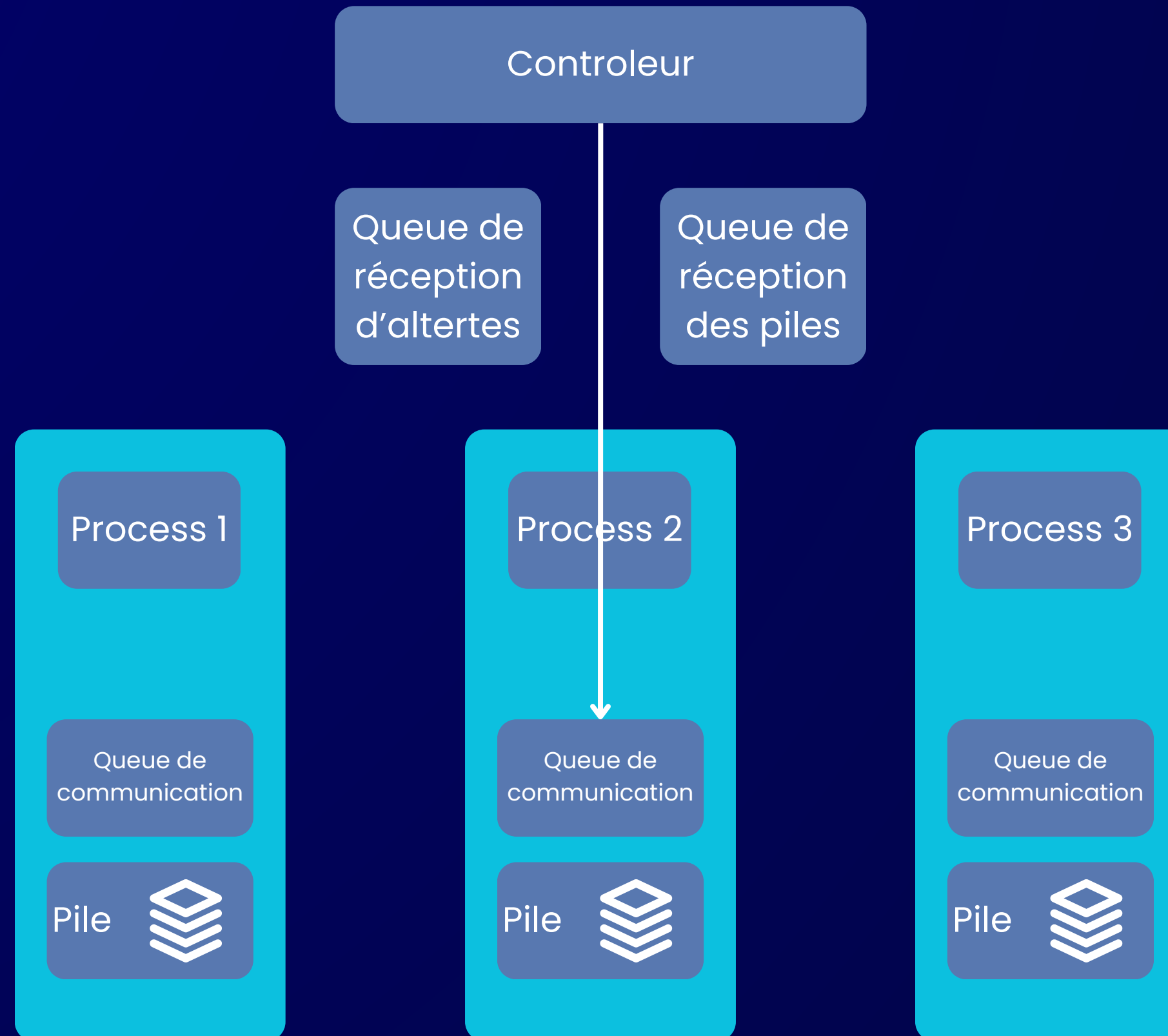
Multiprocessing

La pile est envoyée dans la queue de réception des piles



Multiprocessing

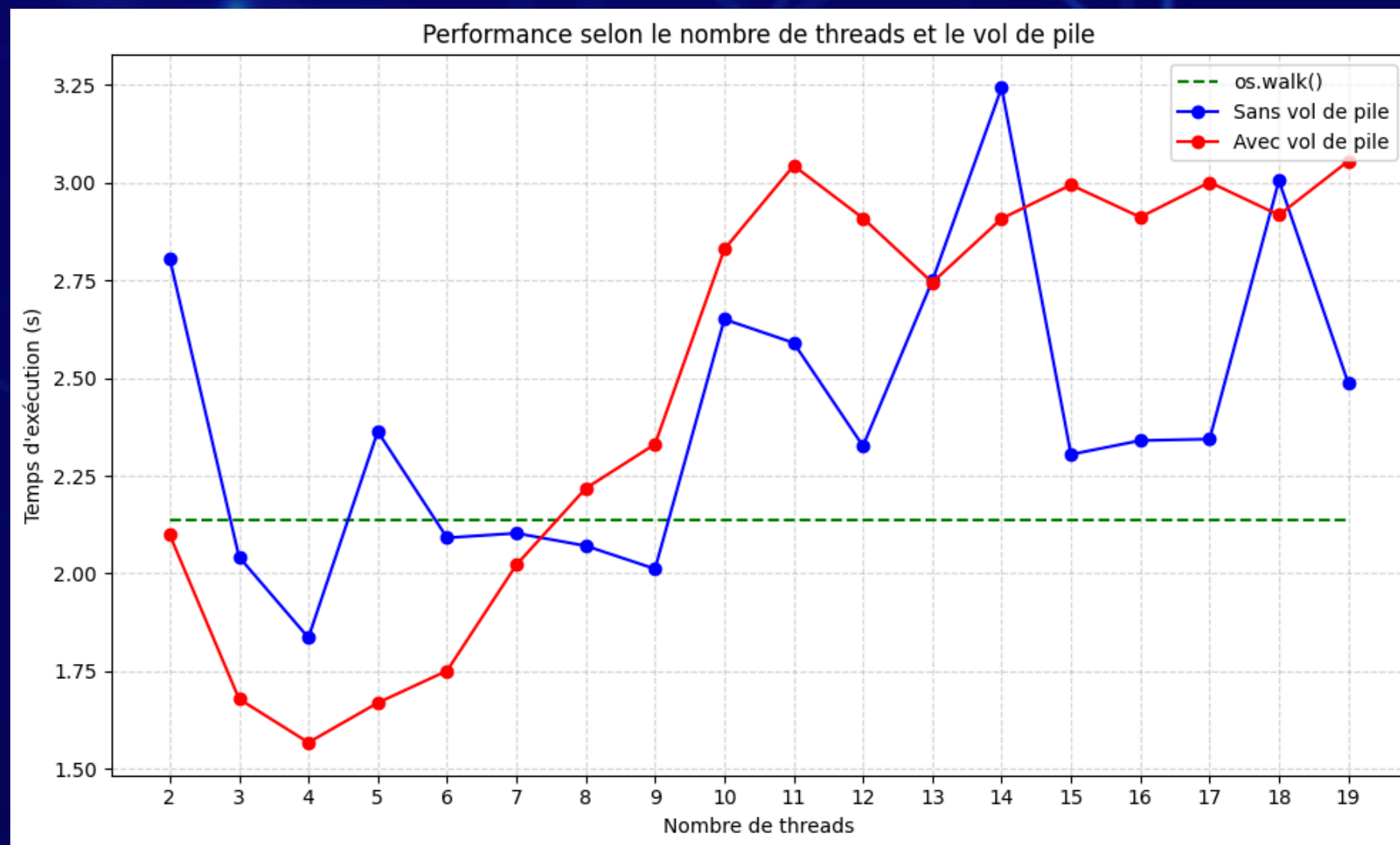
Le controleur renvoie la pile
au process 2, qui continue
sa tache sur la nouvelle pile



Multiprocessing

- Avantages :
 - Avoir plusieurs piles de communication permet de ne pas se perdre et de recevoir des informations simultanées
 - Le contrôleur est celui qui modère et gère les demandes des processus
- Désavantages :
 - Attente de messages, beaucoup d'opérations intermédiaires
 - Perte de temps. Pour optimiser : deque() au lieu des listes

Benchmark – Threads VS Work-StealingThreads



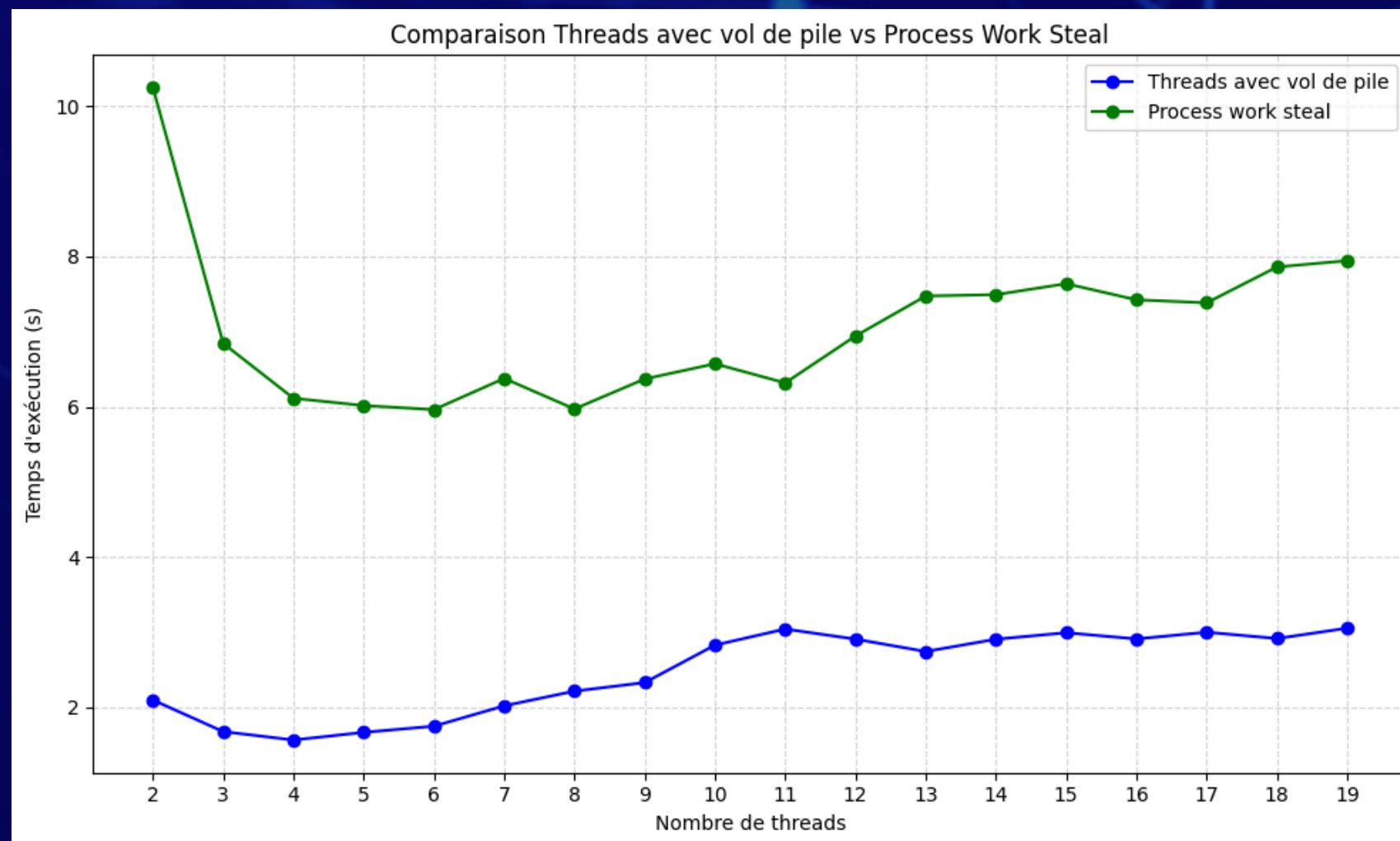
299 748 Fichiers
29 217 Dossiers
16Go de données

Temps os.walk() : 2,134sec

Benchmark – Work–Stealing

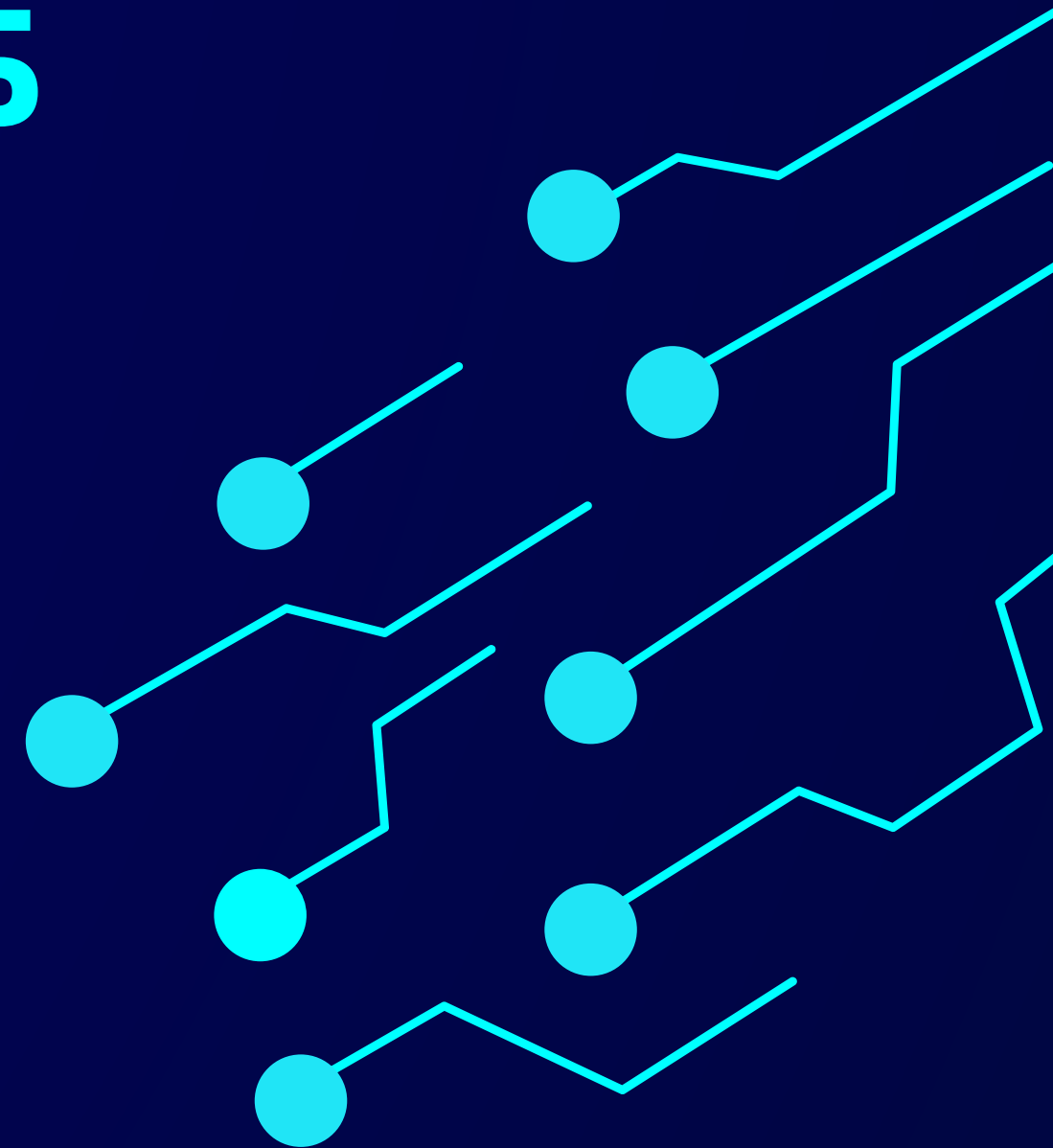
Process vs Work–Stealing Threads

Page 10



CONCLUSION ET PERSPECTIVES

- Micro-optimisations liées à Python.
- Complexifier la tâche du crawler pour le rendre plus pertinent.



MERCI !!!